

Chapitre 3

Algorithmique

Sommaire.

1	Introduction.	1
1.1	Conception : spécifier un problème.	2
1.2	Correction : critères de vérification.	2
1.3	Efficacité : vitesse d'exécution, place mémoire occupée.	2
1.4	Pourquoi s'embêter avant de programmer.	3
1.4.1	Terminaison.	3
1.5	Complexité temporelle.	3
2	Correction totale.	3
2.1	Terminaison.	3
2.1.1	Variant de boucle.	3
2.1.2	Appels récursifs.	4
2.2	Correction.	4
2.2.1	Invariant de boucle.	4
3	Complexité temporelle.	4
3.1	Notation de Landau.	4
3.1.1	Définitions.	4
3.1.2	Des qualificatifs pour les ordres de grandeur.	5
3.2	Comparaison et calculs.	6
3.3	Comment considérer les entrées de taille n ?	7
3.4	Complexité temporelle : quelques exemples de calculs.	7
3.4.1	Schéma général.	7
3.4.2	Tri bulle.	8
3.4.3	Recherche dichotomique.	8
4	Visions plus globales de la complexité.	9
4.1	Complexité en moyenne.	9
4.2	Complexité amortie.	9

Les propositions marquées de ★ sont au programme de colles.

1 Introduction.

Définition 1: Algorithmique.

L’**algorithmique** est la branche de l’informatique qui étudie la conception et l’analyse des algorithmes. Elle comporte trois axes d’étude :
↔ La conception.
↔ La correction.
↔ L’efficacité.

Définition 2: Algorithme.

Un **algorithme** est une méthode systématique de résolution de problèmes.
C'est une suite d'opérations élémentaires ne dépendant que des données en entrée, permettant de donner une réponse à une question posée.
Quand le calcul se termine, la réponse doit être correcte quelle que soit l'entrée fournie.

1.1 Conception : spécifier un problème.**Définition 3: Entrée.**

L'**entrée** d'un algorithme est la nature des données sur lesquelles il est appliqué.

Définition 4: Sortie.

La **sortie** d'un algorithme est la description du résultat que l'algorithme doit fournir.

Définition 5: Problème de décision.

Quand la réponse attendue à une question est **oui** ou **non**, on parle de **problème de décision**.

Définition 6: Décidabilité.

On dit d'un problème de décision qu'il est **décidable** s'il est possible de trouver un algorithme qui le résout.

1.2 Correction : critères de vérification.**Définition 7: Arrêt.**

On dit qu'un algorithme **s'arrête** si le calcul se termine en un temps fini, quelle que soit l'entrée.

Définition 8: Correction.

Un algorithme est **correct** si sa réponse est correcte pour toute entrée pour laquelle son calcul se termine.

Définition 9: Correction partielle.

Si l'algorithme est correct mais ne s'arrête pas sur toutes les entrées, il a une **correction partielle**, sinon **totale**.

1.3 Efficacité : vitesse d'exécution, place mémoire occupée.**Définition 10: Complexité temporelle.**

La **complexité temporelle** d'un algorithme est son temps d'exécution, en général en fonction de la taille de l'entrée.

Définition 11: Complexité spatiale.

La **complexité spatiale** d'un algorithme est la place mémoire occupée en plus de l'entrée pendant le calcul, en fonction de la taille de l'entrée.

1.4 Pourquoi s'embêter avant de programmer.**1.4.1 Terminaison.****Théorème 12: Problème de l'arrêt.**

Il ne peut pas exister d'algorithme avec correction totale permettant de décider si un algorithme donné s'arrête pour une entrée donnée.

Preuve :

Supposons l'existence d'un algorithme **Arrêt** : Algorithme a , entrée $e \mapsto$ Vrai s'il s'arrête, Faux sinon.

On considère l'algorithme suivant : **Paradoxe** : Algorithme $a \mapsto$ si **Arrêt**(a, a) alors boucle infinie, sinon fin.

Si **Paradoxe**(**Paradoxe**) se termine, alors **Arrêt**(**Paradoxe**, **Paradoxe**) est faux, ce qui est contradictoire.

Si **Paradoxe**(**Paradoxe**) ne se termine pas, alors **Arrêt**(**Paradoxe**, **Paradoxe**) est vrai, ce qui est contradictoire.

Donc **Arrêt** ne peut pas exister.

1.5 Complexité temporelle.**Définition 13**

Pourquoi ne pas se contenter d'améliorations matérielles ?

Pour un algorithme de complexité temporelle en n^3 , multiplier la taille de l'entrée par 10 multiplie le temps d'exécution par 1000.

Pour un algorithme de complexité temporelle en n^2 , multiplier la taille de l'entrée par 10 multiplie le temps d'exécution par 100.

Cela ne peut pas être compensé par des améliorations matérielles.

2 Correction totale.**2.1 Terminaison.****Méthode : Cas facile : pas de boucles ou d'appels récursifs.**

Dans ce cas, le nombre de chemins dans le graphe de flot de contrôle est fini.

Si les appels à d'autres algorithmes se terminent, alors la terminaison de l'algorithme est assurée.

2.1.1 Variant de boucle.**Définition 14: Variant.**

Un **variant de boucle** est une expression qui prend des valeurs entières positives dont la valeur diminue strictement à chaque itération.

Si on est assurés de la terminaison de chaque itération dans une boucle, alors l'existence d'un variant de boucle nous assure la terminaison.

⚠ Un variant ne diminue pas forcément jusqu'à 0, il suffit qu'il soit minoré par une constante.

2.1.2 Appels récursifs.

Définition 15: Appel récursif.

On appelle **appel récursif** pour une fonction le fait de s'appeler elle-même.

Définition 16: Variant d'appel.

Un **variant d'appel** est une expression qui prend des valeurs entières positives et dont la valeur diminue strictement à chaque appel récursif.

Si on est assurés que les autres calculs de l'algorithme se terminent, alors l'existence d'un variant d'appel nous assure qu'on sort de la fonction.

2.2 Correction.

2.2.1 Invariant de boucle.

Définition 17: Invariant.

Un **invariant de boucle** est un prédicat.

C'est une expression booléenne qui vérifie les propriétés suivantes :

↔ Vrai avant d'entrer dans la boucle.

↔ S'il est vrai à l'entrée d'une itération et qu'on sort de l'itération, alors il est vrai en sortie.

Si on sort de la boucle, les propriétés de l'invariant de boucle assurent qu'il est encore vrai.

Méthode : Cas des fonctions récursives.

La preuve de la correction des fonctions récursives se fait en général par récurrence sur la taille de l'entrée.

On peut supposer que les appels récursifs sur des entrées de taille strictement inférieure se terminent et donnent une réponse correcte.

Définition 18: Pas facile de trouver un invariant.

En 1951, Rice a démontré qu'aucune propriété sémantique n'est décidable. Une des conséquences est qu'il n'y a pas de méthode générale pour décider un invariant.

3 Complexité temporelle.

3.1 Notation de Landau.

3.1.1 Définitions.

On conseille de voir le chapitre 29 de mathématiques pour plus de détails.

Soient f et g deux fonctions de $\mathbb{N}$ dans $\mathbb{R}_+$.

Définition 19: Grand O.

On dit que f est en grand O de g , et on note $f \in O(g)$, si :

$$\exists C > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, f(n) \leq C \cdot g(n).$$

Définition 20: Grand Omega.

On dit que f est en grand Omega de g , et on note $f \in \Omega(g)$ si :

$$\exists C > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, f(n) \geq C \cdot g(n).$$

Définition 21: Grand Theta.

On dit que f est en grand Theta de g , et on note $f \in \Theta(g)$ si :

$$\exists C, D > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, C \cdot g(n) \leq f(n) \leq D \cdot g(n).$$

Proposition 22: Trivial.

$$f \in \Theta(g) \iff [f \in O(g) \text{ et } f \in \Omega(g)] \iff [f \in O(g) \text{ et } g \in O(f)]$$

Proposition 23: Symétrie.

La relation Θ est symétrique : $f \in \Theta(g) \iff g \in \Theta(f)$.

3.1.2 Des qualificatifs pour les ordres de grandeur.

Définition 24

On dit d'une fonction en $\Omega(n)$ qu'elle est au moins linéaire.
Si elle est en $O(n)$, elle est au plus linéaire, si elle est en $\Theta(n)$, elle est linéaire.

Définition 25

Notation	Nom	Exemple
$O(1)$	Constante	$1, 2, 3, \dots$
$O(\log n)$	Logarithmique	$\log n, \log_2 n, \log_{10} n$
$O(\sqrt{n})$	Racinaire	$\sqrt{n}, \sqrt{2n}, \sqrt{3n}, \dots$
$O(n)$	Linéaire	$n, 2n, 3n, \dots$
$O(n \log n)$	Linéarithmique	$n \log n, 2n \log n, 3n \log n, \dots$
$O(n^2)$	Quadratique	$n^2, 2n^2, 3n^2, \dots$
$O(n^3)$	Cubique	$n^3, 2n^3, 3n^3, \dots$
$O(n^k)$	Polynomiale	$n^k, 2n^k, 3n^k, \dots$
$O(a^n)$	Exponentielle	$2^n, 2^{n+1}, 2^{n+2}, \dots$
$O(n!)$	Factorielle	$n!, (n+1)!, (n+2)!, \dots$

3.2 Comparaison et calculs.

Définition 26

Soient deux fonctions f et g allant de $\mathbb{N}$ vers $\mathbb{R}_+$, ne convergeant pas vers 0.

$$\frac{f(n)}{g(n)} \xrightarrow{n \rightarrow +\infty} l \in \mathbb{R} \implies f \in O(g)$$

De plus, $f \in \Theta(g) \iff l \neq 0$.

Justification :

Supposons que f/g converge vers $l \in \mathbb{R}$. Toute suite convergente est bornée donc $\exists C \in \mathbb{R}_+^*$, $f(n) < C \cdot g(n)$.

$\implies$ Supposons par contraposée que $l = 0$. Alors g/f diverge vers $+\infty$ comme inverse.

$$\forall \varepsilon > 0, \exists N \in \mathbb{N}, \forall n \geq N, g(n) > \varepsilon \cdot f(n) \text{ donc } g \notin O(f) \text{ donc } f \notin \Theta(g).$$

$\impliedby$ Supposons $l \neq 0$. Alors g/f converge, donc $g \in O(f)$ (et $f \in O(g)$) donc $f \in \Theta(g)$.

Proposition 27

Soient f_1, f_2, g_1, g_2 de $\mathbb{N}$ dans $[1, +\infty[$ tels que $f_1 \in O(g_1)$ et $f_2 \in O(g_2)$. Alors:

$$f_1 + g_1 \in O(g_1); \quad f_1 + f_2 \in O(g_1 + g_2); \quad f_1 f_2 \in O(g_1 g_2).$$

Proposition 28

Soient f, g, h de $\mathbb{N}$ dans $]1, +\infty[$ tels que $f \in O(g)$ et h non bornée et croissante apdcr.
Alors $f \circ h \in O(g \circ h)$.

Preuve :

Puisque $f \in O(g)$, on a $\exists C > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, f(n) < C \cdot g(n)$.

Puisque h est croissante et non bornée : $\exists n_1 \in \mathbb{N}, \forall n \geq n_1, h(n) \geq n$.

Alors $\forall n \geq \max(n_0, n_1), f(h(n)) < C \cdot g(h(n))$ donc $f \circ h(n) < C \cdot g \circ h(n)$.

Corrolaire 29

Soient $k > l \geq 0$. On a $n^l \in O(n^k)$ et $n^l \notin \Theta(n^k)$.

Corrolaire 30

Soit $k \in \mathbb{N}$. Toute fonction polynomiale associée à un polynôme de degré k et de coefficient dominant non nul est d'ordre de grandeur $\Theta(n^k)$.

Corrolaire 31

Soit $k \geq 1$ fixé. Alors $k^n \in O((k+1)^n)$ et $k^n \notin \Theta((k+1)^n)$

Proposition 32

$$\forall k \in \mathbb{N}, (\log n)^k \in O(n) \quad \text{et} \quad (\log n)^k \notin \Theta(n).$$

Corrolaire 33

$$\forall k \in \mathbb{N}, n^k \in O(2^n) \quad \text{et} \quad n^k \notin \Theta(2^n).$$

3.3 Comment considérer les entrées de taille n ?

Notation locale : pour $k \in \mathbb{N}$, on note E_n l'ensemble des entrées de taille n pour un algorithme donné.

Définition 34

Soit un algorithme A possédant une correction totale. On note c_A la fonction qui à une instance E de l'entrée de A associe le nombre d'opérations élémentaires nécessaires au déroulement de A sur E .

Définition 35

On appelle **complexité dans le pire des cas** de A la fonction $C_A : n \mapsto \max_{E \in E_n} (c_A(E))$.
C'est une garantie de complexité, on ne fera pas pire quelle que soit l'entrée, même si la complexité dans le pire des cas est mauvaise, en pratique, l'algorithme peut être utilisable.

Définition 36

On appelle **complexité dans le meilleur des cas** de A la fonction $C_A : n \mapsto \min_{E \in E_n} (c_A(E))$.
Elle est toujours inférieure à la complexité dans le pire des cas. Si elles sont égales, la complexité est la même sur toute entrée.

3.4 Complexité temporelle : quelques exemples de calculs.**3.4.1 Schéma général.****Méthode : Dans le pire des cas.**

La plupart du temps, on suit le schéma suivant :
 $\hookrightarrow$ Trouver un majorant significatif de la complexité.
 $\hookrightarrow$ Trouver une famille d'entrées pour laquelle la complexité est minorée par $\alpha f(n)$, pour α constante.
 On conclut à $\Theta(f)$.

3.4.2 Tri bulle.

Définition 37

Algorithme 1 : Tri à bulles.

Entrées : Liste de réels L .

Sorties : L triée dans l'ordre croissant.

permutation $\leftarrow$ Vrai.

tant que permutation **faire**

 permutation $\leftarrow$ Faux.

pour i allant de 0 à $n - 1$ **faire**

si $L[i] > L[i + 1]$ **alors**

 Échanger $L[i]$ et $L[i + 1]$.

 permutation $\leftarrow$ Vrai.

fin

fin

$n \leftarrow n - 1$.

fin

Opérations élémentaires :

$\hookrightarrow$ Comparaisons : à chaque fois.

$\hookrightarrow$ Affectations : pas toujours.

On ne compte que les comparaisons, pas les affectations.

Proposition 38

La complexité temporelle dans le meilleur des cas du tri à bulles est en $\Theta(n)$.

Preuve :

Dans le meilleur des cas, on ne fait qu'une itération dans la boucle tant que, donc on fait n comparaisons.

On peut donc minorer la complexité par n . On peut la majorer par n dans le cas où la liste en entrée est triée.

On a donc bien une complexité en $\Theta(n)$.

3.4.3 Recherche dichotomique.

Définition 39: Algorithme de la recherche dichotomique.

Algorithme 2 : Recherche dichotomique.

Entrées : Liste L de taille n d'éléments comparables triée dans l'ordre croissant, et un élément x .

Sorties : Un booléen indiquant si $x \in L$.

milieu $\leftarrow L[n/2]$ **si** milieu $== x$ **alors**

retourner Vrai.

fin

sinon si milieu $> x$ **alors**

retourner dichotomie($L[0 : n/2]$, x , $n/2$)

fin

retourner dichotomie($L[n/2 + 1 : n]$, x , $n/2$)

où $L[i : j]$ est la liste des éléments de L d'indices dans $\llbracket i, j \rrbracket$.

Proposition 40

On s'intéresse à l'algorithme de recherche dichotomique dans une liste triée.

La complexité temporelle dans le meilleur des cas est en $\Theta(1)$.

La complexité temporelle dans le pire des cas est en $\Theta(\log n)$.

Preuve :

On note N la taille du tableau d'origine, et $T(n)$ le nombre d'opérations élémentaires sur un tableau de taille n .

On a la relation de récurrence suivante : $T(0) = 1$ et pour $n > 0$, $T(n) = \alpha + T(n/2)$.

Pour simplifier les calculs, on suppose que $n = 2^k$. Alors $T(n) = T(2^k) = \alpha + T(2^{k-1}) = \dots = k\alpha + T(1)$.

Cas général : $\forall n \in \mathbb{N}, \exists k \in \mathbb{N} \ 2^k \leq n \leq 2^{k+1}$ donc $T(n) = O(k) = O(\log_2 n)$.

Si l'élément à trouver dans le tableau n'y est pas, on a une complexité en $\Omega(\log n)$, d'où le $\Theta(\log n)$.

4 Visions plus globales de la complexité.

4.1 Complexité en moyenne.

Définition 41

Soit un algorithme A avec correction totale et $\mathcal{E}_n$ l'ensemble de ses entrées de taille n .

On note c_A qui à une entrée E associe le nombre d'opérations élémentaires au déroulement de A sur E . On appelle complexité en moyenne de A la fonction :

$$c_A^{moy} : n \mapsto \frac{1}{|\mathcal{E}_n|} \sum_{E \in \mathcal{E}_n} c_A(E).$$

En supposant un tirage uniforme sur les entrées d'une taille donnée.

Exemple 42

Pour un exemple de calcul de complexité en moyenne, voir le cas du tri rapide dans le chapitre 4 - Collections de données, proposition 36.

4.2 Complexité amortie.

Définition 43

L'analyse amortie consiste à relativiser le coût élevé de certaines opérations réalisées sur une structure de données.

Au lieu d'étudier les opérations de manière indépendante, on les étudie en tant que suite d'opérations dont on aimerait connaître la complexité moyenne.

Cela permet notamment d'exhiber que la rapidité de certaines opérations est la conséquence de la lenteur d'autres.

Par exemple, pour la structure Unir-Et-Trouver (MPI) on a des implémentations où on favorise l'union, et d'autres où on favorise la recherche du représentant.

Méthode : Agrégats.

On étudie une suite d'opérations, et on observe la moyenne de la complexité d'une opération.

Exemple 44: Tableaux dynamiques.

On aimerait avoir une structure dans laquelle on peut insérer un élément à la fin et accéder au $i^{\text{ème}}$ élément. Un tableau dynamique est composé de trois valeurs :

↔ Une taille $n \in \mathbb{N}$, une capacité totale $m \in \mathbb{N}$ et un tableau de taille m .

Lorsqu'on ajoute un élément, on l'ajoute à la case n du tableau, on incrémente n . Si n atteint m , alors le tableau est plein : on en crée un nouveau de taille αm ($\alpha \in \mathbb{R}$), et on y copie les valeurs de l'ancien tableau.

Si on a un tableau dynamique dans lequel on insère k éléments, à chaque fois que n est de la forme α^i , on doit faire une insertion lente. Chaque insertion lente est moins rapide que la précédente, mais elles se font de plus en plus rares. Si on part d'un tableau dynamique vide, on a des insertions lentes pour $i \in \{j \in \mathbb{N}, \alpha^j \leq k\}$.

On note c_i le coût de la $i^{\text{ème}}$ opération :

$$\frac{\sum_{i=1}^k c_i}{k} \leq \frac{\sum_{i=1}^k 1 + \sum_{j=0}^{\lfloor \log_{\alpha} k \rfloor} \alpha^j}{k} \leq 1 + \frac{\alpha^{\lfloor \log_{\alpha} k \rfloor + 1}}{k} - \frac{1}{(\alpha - 1)k} \leq 1 + \frac{\alpha k}{k} = O(1)$$

Méthode : Potentiels.

On crée une fonction Φ et celle-ci nous renvoie une valeur selon l'état de la structure.

Alors le coût amorti d'une opération est égal à son coût réel moins la variation de Φ après son exécution.

Exemple 45

Pour le tableau dynamique, $\Phi = m - n$ quand on réalise une insertion rapide, $\Delta\Phi = -1$, et pour une insertion lente, $\Delta\Phi = (\alpha - 1)n - 1$. Dans les deux cas, le coût amorti est en $O(1)$.

Méthode : Banquier.

On paie un coût fictif à chaque opération, qui dépend uniquement de l'opération choisie. Les coûts en trop serviront à financer les opérations qui ont un coût réel plus élevé que leur coût fictif.

Exemple 46

On a envie de fixer un coût permettant le financement des insertions lentes. L'insertion lente consiste à créer un tableau de taille αn et à recopier les n premières cases. On veut donc avoir $\alpha n + n$ pièces dans la structure de données lors d'une insertion lente, donc que les n insertions rapides précédentes aient laissé $n\alpha + n$ pièces.

✗ Un coût de $\alpha + 1$ ne fonctionne pas car lors d'une insertion lente, les n valeurs réinsérées ne contribuent pas au financement.

✓ Un coût de $\alpha + 1 + k$ avec $k = \frac{\alpha+1}{\alpha-1}$ fonctionne car il compense le déficit créé par ces n valeurs :

$$(\alpha n - n)(\alpha + 1 + k) \geq \alpha n(\alpha + 1) \implies k(\alpha - 1) \geq \alpha + 1 \implies k \geq \frac{\alpha + 1}{\alpha - 1}.$$

Le coût amorti d'une opération est alors le coût fictif de celle-ci.

Ici, toutes les insertions ont un coût amorti en $O(\alpha + 1 + k) = O(1)$.

← FIN →